Optimasi Alokasi Dana Saham LQ45 dengan Batas Risiko dan Kardinalitas

Latar Belakang

Alokasi dana investor ritel ke saham LQ45 adalah masalah kombinatorial, bukan masalah kontinu biasa, karena dana terbatas, saham dibeli per lot, dan jumlah saham yang dipegang dibatasi. Indeks LQ45 memuat 45 saham berlikuiditas tinggi di Bursa Efek Indonesia, sehingga menjadi universe yang wajar untuk keputusan seperti ini.

Kerangka mean-variance Markowitz (1952) menegaskan bahwa return dan risiko portofolio harus ditimbang bersama. Dalam praktik, keputusan itu dibatasi beberapa hal: jumlah saham yang dipegang tidak banyak, satu saham tidak boleh mendominasi, dan pembelian hanya bisa dalam kelipatan lot (100 lembar).

Batas jumlah saham (kardinalitas) dan lot bulat mengubah masalah kontinu yang mudah menjadi masalah kombinatorial. Masalah mean-variance dengan batas kardinalitas diketahui NP-hard (Bienstock, 1996), dan literatur memakai metaheuristik seperti Genetic Algorithm (GA) dan Simulated Annealing (SA) untuk menanganinya (Chang dkk., 2000).

Dua pelajaran dari tahap awal proyek ini membentuk rumusan sekarang. Pertama, memaksimalkan return saja tidak menarik: dengan batas 20% per saham, jawabannya hanya mengurutkan saham menurut return historis dan sama untuk semua $K \geq 5$. Kedua, pada formulasi kontinu tanpa kardinalitas (maksimasi Sharpe, 42 saham), GA hanya mereproduksi solusi solver konveks dengan selisih bobot L1 sebesar 0,006, sehingga perbandingan algoritma tidak memberi informasi.

Penelitian ini karena itu memaksimalkan return ekspektasi di bawah batas risiko portofolio, dengan batas jumlah saham K (2, 3, 5, 7, dan 10), batas 20% per saham untuk K = 7 dan 10, dan pembelian dalam lot. Dengan batas risiko, pemilihan saham bergantung pada kovarians (diversifikasi), bukan hanya pada return masing-masing, sehingga pilihan K bermakna. Algoritma pencarian benar-benar diuji pada 7 dan 10 saham, sedangkan pada 5 saham atau kurang ruang pencariannya cukup kecil untuk dienumerasi penuh. GA, SA, PSO, dan Differential Evolution (DE) dibandingkan dengan pembanding eksak dan heuristik sederhana, lalu dinilai di dalam sampel (kualitas algoritma) dan di luar sampel (kualitas strategi).

Rumusan Masalah

Penelitian ini menjawab empat masalah alokasi dana yang saling terkait.

1. **Kardinalitas dan konsentrasi.** Memilih sejumlah saham tertentu (paling banyak 2, 3, 5, 7, atau 10 saham) dari 42 saham LQ45. Batas 20% dana untuk satu saham hanya berlaku pada pilihan 7 dan 10 saham.
2. **Alokasi dana tertentu.** Membagi dana awal yang sudah ditetapkan ke saham terpilih; sisa dana yang tidak terpakai (kas) diusahakan sekecil mungkin.

3. **Kelipatan lot.** Jumlah pembelian tiap saham harus kelipatan 1 lot (100 lembar), sehingga porsi dana tiap saham ditentukan oleh harga saham dan jumlah lot.
4. **Batas risiko.** Fluktuasi nilai portofolio (volatilitas) tidak boleh melebihi batas yang ditetapkan dari risiko portofolio pembanding yang dananya dibagi sama rata.

Pertanyaan penelitian:

1. Berapa return ekspektasi tertinggi yang dapat dicapai jika investor hanya boleh memilih 2, 3, 5, 7, atau 10 saham, pada satu tingkat batas risiko, dan bagaimana return itu berubah ketika jumlah saham dikurangi?
2. Seberapa dekat hasil Genetic Algorithm (GA), Simulated Annealing (SA), Particle Swarm Optimization (PSO), dan Differential Evolution (DE) dengan hasil solusi eksak, dilihat dari selisih return terhadap solusi eksak, waktu komputasi, dan kestabilan hasil antar pengulangan?
3. Apakah hasil yang baik pada data historis (data latih) tetap baik pada data periode berikutnya (data uji), dan apakah batas risiko tetap terpenuhi pada data uji?

Tujuan, Batasan, dan Asumsi

Tujuan utamanya adalah memaksimalkan return ekspektasi di bawah batas risiko, lalu membandingkan algoritma pencarinya secara adil.

Tujuan

1. Memaksimalkan return ekspektasi portofolio saham LQ45 dengan menjaga risiko (volatilitas) tidak melebihi batas yang ditetapkan.
2. Menentukan kombinasi saham dan jumlah lot terbaik untuk setiap pilihan jumlah saham.
3. Membandingkan GA, SA, PSO, DE, heuristik sederhana (greedy forward selection, serta pemilihan saham dengan return tertinggi lalu mengoptimalkan porsinya), dan solusi eksak dengan anggaran evaluasi yang sama.
4. Menguji kinerja strategi pada data periode berikutnya dengan metode walk-forward, termasuk apakah batas risiko tetap terpenuhi.

Batasan masalah

- Universe: 42 saham LQ45 dengan data cukup (AMMN, GOTO, MBMA, dan PGEO dibuang), periode November 2021 sampai September 2026.
- Hanya posisi beli (long-only), tanpa short selling dan tanpa leverage.
- Strategi buy-and-hold di dalam tiap jendela uji, tanpa rebalancing di dalam jendela.
- Daftar saham adalah konstituen LQ45 saat ini, sehingga ada survivorship bias yang harus dinyatakan sebagai keterbatasan.
- Risiko diukur dengan volatilitas (deviasi baku), bukan CVaR.

Asumsi awal (bisa diubah sesuai kebutuhan tugas)

Asumsi	**Nilai awal**	**Alasan**
Dana D	Rp 5 juta	Sesuai modal mahasiswa; membuat kendala lot sangat bermakna (untuk K = 7 dan 10, batas 20% = Rp 1 juta per saham, sehingga 3 saham LQ45 dengan 1 lot di atas Rp 1 juta tidak dapat dibeli). Sensitivitas terhadap D tidak diuji pada tahap ini (pengembangan lanjutan)
Kas	Diminimalkan: sisa dana ≤ 5% dari D (kendala lunak, selalu dilaporkan); return kas = risk-free	Dana harus terpakai. Karena batas 20% hanya untuk K = 7 dan 10, semua nilai K dapat terinvestasi penuh. Pada K kecil, sisa dana bergantung pada harga lot (mis. K = 5 pada Rp 5 juta hanya sekitar 15% himpunan saham acak yang dapat mencapai kas ≤ 5%)
Biaya transaksi	Tidak dimodelkan pada versi dasar	Disederhanakan; menjadi ekstensi opsional
Dividen	Tercermin lewat harga adjusted (total return)	Mencegah penghitungan ganda
Risk-free	5% per tahun	Tetap; sensitivitas opsional dengan suku bunga acuan
Horizon uji	126 hari bursa (sekitar 6 bulan)	Dasar walk-forward

Formulasi Matematis

Masalahnya memilih jumlah lot n_i tiap saham agar return ekspektasi maksimum, dengan risiko tidak melebihi σ_max, paling banyak K saham, dan, khusus K = 7 dan 10, tiap saham paling banyak 20% dana.

Notasi

Simbol	Arti
N = 42	Jumlah saham kandidat
D	Dana awal (Rp)
P_i	Harga beli saham i per lembar (harga tanpa penyesuaian)
n_i	Jumlah lot saham i, bilangan bulat ≥ 0 (1 lot = 100 lembar)
z_i	1 bila saham i dipilih, 0 bila tidak
w_i	Bobot saham i terhadap dana
w_0	Bobot kas
μ, Σ	Return ekspektasi tahunan dan kovarians tahunan (dari data latih)
r_f	Return kas (risk-free)
K, σ_max	Batas jumlah saham dan batas volatilitas

Bobot dan fungsi objektif

$$wi = \frac{100 n_i P_i}{D}, \; w_0 = 1 - \sum_{i-1}^{N} wi$$

$$\max_{n,z} R = \sum_{i=1}^{N} w_i \boldsymbol{\mu}_I + w_0 r_f$$

Kendala

$$\sqrt{w^T \sum w} \leq \sigma_{max} \qquad (batas\ risiko)$$

$$0 \leq w_i \leq C_K \qquad (batas\ per\ saham)$$

$$w_0 \geq 0 \qquad (dana\ tidak\ terlampaui)$$

$$\sum_i z_i \leq K,\ n_i \leq M z_i \qquad (Kardinalitas)$$

$$z_i \in \{0, 1\},\ n_i \in \mathbb{Z} \geq 0 \qquad (lot\ bulat)$$

Posisi minimum ($n_i \geq 1$ bila $z_i = 1$) dapat ditambahkan agar tidak ada posisi yang terlalu kecil. Batas per saham c_K bernilai 20% untuk K = 7 dan 10, dan tidak ada (100%) untuk K = 2, 3, dan 5. Pada $K \leq 5$, batas 20% membuat dana tidak mungkin terinvestasi penuh (K = 5 terkunci pada bobot sama rata, $K < 5$ tidak layak), sehingga konsentrasi dibatasi oleh K dan batas risiko.

Mengapa masalah ini tidak trivial. Tanpa kendala risiko, objektifnya linear dan jawabannya cukup mengurutkan saham menurut μ. Dengan kendala risiko, saham terbaik bergantung pada kovarians antar-saham, jadi himpunan terbaik harus dicari, bukan diurutkan.

Ukuran ruang pencarian pemilihan saham (jumlah himpunan saham yang mungkin, C(42, K))

K	**Jumlah himpunan**
1	42
2	861
3	11.480
4	111.930

5	850.668
7	26.978.328
10	1.471.442.973

Untuk K kecil, semua himpunan masih bisa dienumerasi; untuk K = 7 dan 10, pencarian menyeluruh sudah berat. Karena itu perbedaan antar-algoritma kemungkinan paling terlihat pada K besar, dan ini diperlakukan sebagai hipotesis yang diuji, bukan klaim.

Data dan Praproses

Data harian 42 saham LQ45 dari repo yang sudah ada dipakai ulang, dengan dua perbaikan: definisi return disatukan dan harga tanpa penyesuaian ditambahkan untuk perhitungan lot.

Sumber dan cakupan. Harga harian diunduh dari yfinance (ticker berakhiran .JK). Setelah pembersihan tersisa 42 saham dan sekitar 1.167 hari bursa (23 November 2021 sampai 30 September 2026).

Dua jenis harga, dua fungsi.

Jenis harga	**Dipakai untuk**	**Catatan**
Adjusted close	Return historis, μ, Σ, dan return realisasi saat evaluasi	Sudah memuat dividen dan stock split, jadi mendekati total return
Close tanpa penyesuaian	Harga beli P_i untuk menghitung jumlah lot dan nilai pembelian	Perlu diunduh ulang dengan auto_adjust=False

Dividen tidak ditambahkan lagi sebagai data terpisah selama return berasal dari adjusted close; menambahkannya akan menghitung dividen dua kali. Pemodelan dividen sebagai arus kas tunai menjadi ekstensi opsional.

Return sederhana, bukan log. Optimasi dan evaluasi memakai definisi yang sama, yaitu return sederhana harian. Pada tahap awal proyek, μ dari log return lebih rendah rata-rata 7,6% per tahun daripada μ aritmetik, sehingga Sharpe yang dioptimasi dan Sharpe yang dievaluasi tidak sebanding.

Estimasi μ dan Σ. Versi dasar memakai rata-rata return harian × 252 dan kovarians sampel × 252 pada jendela latih. Analisis sensitivitas memakai shrinkage μ ke rata-rata lintas-saham dan kovarians Ledoit-Wolf.

Ketidakpastian μ besar. Pada tahap awal (3,66 tahun data latih), galat baku μ per saham bernilai median sekitar 18,7% per tahun dan tidak ada saham dengan $|t| > 2$. Karena itu return ekspektasi hanya dipakai sebagai alat optimasi, dan klaim kinerja harus berasal dari data uji.

Pembagian data (walk-forward).

1. Jendela latih 504 hari bursa (sekitar 2 tahun) untuk menghitung μ, Σ, dan σ_{max}.
2. Alokasi dibeli pada hari pertama jendela uji dengan harga tanpa penyesuaian, lalu ditahan 126 hari bursa (sekitar 6 bulan).
3. Jendela digeser 126 hari; pada data saat ini dapat terbentuk 5 jendela; penelitian ini memakai 3 jendela (jendela terakhir yang memiliki data latih lengkap).
4. Fase perbandingan algoritma memakai seluruh data latih sekaligus (923 hari) sebagai satu kasus tetap.

Risk-free. Nilai awal 5% per tahun. Bila waktu mengizinkan, diganti dengan suku bunga acuan atau yield SBN per jendela.

Parameter Penelitian

Parameter dibagi dua: parameter masalah yang sama untuk semua algoritma, dan parameter algoritma yang dipilih sebagai nilai awal lalu disetel ringan hanya pada data latih.

Parameter masalah

Parameter	**Simbol**	**Nilai awal**	**Keterangan**
Dana awal	D	Rp 5.000.000	Satu level; variasi modal tidak diteliti pada tahap ini
Ukuran lot	L	100 lembar	Aturan bursa
Batas per saham	c_{max}	20% (hanya $K = 7$ dan 10)	Dari rumusan masalah
Jumlah saham maksimum	K	2, 3, 5, 7, 10	Lima skenario

Batas risiko	σ_max	1,5× risiko benchmark (satu level)	Benchmark = portofolio 1/N dari 42 saham pada jendela latih
Return kas	r_f	5% per tahun	Tetap pada versi dasar
Posisi minimum	n_i	≥ 1 lot bila dipilih	Mencegah posisi terlalu kecil
Hari bursa per tahun	–	252	Anualisasi
Jendela latih / uji / geser	–	504 / 126 / 126 hari	Walk-forward
Jumlah seed	–	30 (seed 0 sampai 29)	Untuk algoritma stokastik
Anggaran evaluasi fitness	–	20.000 per run	Sama untuk semua metaheuristik

Parameter algoritma (nilai awal; disetel hanya di data latih)

Algoritma	**Parameter**
GA	Populasi 100, generasi 200, tournament k = 3, crossover SBX (probabilitas 0,9; η = 15), mutasi Gaussian (probabilitas 1/N per gen; σ = 0,1), elitisme 2
SA	20.000 iterasi, suhu awal dikalibrasi agar penerimaan awal sekitar 80%, pendinginan geometrik sampai suhu akhir = 0,001 × suhu awal
PSO	Swarm 40, 500 iterasi, inersia 0,7, c1 = c2 = 1,5

DE	Populasi 100, 200 generasi, rand/1/bin, F = 0,5, CR = 0,9
Solver eksak	Enumerasi penuh himpunan saham untuk K = 2, 3, 5; untuk K = 7 dan 10 acuannya solusi terbaik yang ditemukan semua algoritma (MISOCP opsional bila waktu cukup)

Populasi × generasi (GA, DE), swarm × iterasi (PSO), dan jumlah iterasi (SA) semuanya bernilai 20.000 evaluasi, sehingga perbandingan tidak dimenangkan oleh anggaran komputasi yang lebih besar.

Algoritma yang Dibandingkan

Perbandingan hanya adil jika semua algoritma memecahkan masalah yang sama, dengan cara menerjemahkan solusi yang sama, dan dengan anggaran evaluasi yang sama. Karena itu semua metaheuristik memakai satu **decoder bersama** dan satu **fungsi evaluasi bersama**; yang berbeda hanya cara mereka mencari.

1. Pembanding acuan (bukan metaheuristik)

Pembanding	**Fungsi**	**Catatan**
Solver eksak (enumerasi penuh untuk K = 2, 3, 5)	Memberi nilai optimum sejati (ground truth) untuk K = 2, 3, 5	Wajib untuk K = 2, 3, 5 (ruang pencarian 861 sampai 850.668 himpunan). Tanpa ini klaim "kombinasi paling optimal" tidak bisa dibuktikan. Untuk K = 7 dan 10 tidak ada optimum terbukti; acuannya solusi terbaik yang ditemukan semua algoritma.
Heuristik: greedy forward selection dan top-K + QP	Greedy: tambah satu saham per langkah yang paling menaikkan return selama risiko terjaga (aturan Deb). Top-K + QP: pilih K saham dengan μ tertinggi (atau rasio μ/σ), lalu optimalkan bobot dengan QP	Baseline cerdas yang murah. Jika GA tidak mengalahkannya, GA tidak memberi nilai tambah. Top-K + QP sering tidak layak pada batas risiko ketat (1,5×); ketidaklayakannya dilaporkan, bukan disembunyikan.

Acak valid (K saham acak, bobot sama, hanya yang memenuhi kendala)	Baseline naif	Dijalankan 1.000 kali agar punya distribusi, bukan satu titik.

2. Decoder bersama

Setiap kandidat solusi adalah vektor kontinu dengan 2N elemen: N **kunci prioritas** k_i dan N **bobot mentah** v_i, $i = 1..42$. Decoder mengubahnya menjadi portofolio layak secara deterministik:

1. Ambil K saham dengan k_i terbesar sebagai himpunan terpilih S.
2. Normalkan v_i untuk $i \in S$ menjadi bobot awal sehingga jumlahnya $1 - w_0$ (w_0 = porsi kas awal, boleh 0).
3. Terapkan batas per saham 20% (hanya untuk K = 7 dan 10): potong yang melebihi, bagikan sisa ke saham lain yang belum penuh (iterasi sampai stabil). Untuk $K \leq 5$ langkah ini dilewati.
4. Ubah ke lot: $n_i = \text{floor}(w_i \cdot D / (100 \cdot P_i))$; sisa dana kecil masuk kas.
5. Hitung bobot akhir, σ, dan R.

Dengan desain ini kardinalitas K, batas per saham (untuk K = 7 dan 10), dan kelipatan lot **selalu terpenuhi oleh konstruksi** sehingga tidak perlu penalti untuk ketiganya. Satu-satunya kendala yang masih bisa dilanggar adalah batas risiko $\sigma \leq \sigma_{max}$.

Penanganan batas risiko — aturan kelayakan Deb: (a) solusi layak selalu lebih baik daripada solusi tak layak; (b) dua solusi layak dibandingkan lewat R (lebih besar lebih baik); (c) dua solusi tak layak dibandingkan lewat besar pelanggaran $\sigma - \sigma_{max}$ (lebih kecil lebih baik). Cara ini tidak memerlukan koefisien penalti yang harus ditala, berbeda dengan fitness = $R - \lambda \cdot$pelanggaran.

> **Catatan desain.** Memakai kunci prioritas (alih-alih biner z_i) membuat operator kontinu seperti SBX, PSO, dan DE dapat dipakai pada masalah campuran diskrit-kontinu tanpa memodifikasi operatornya. Kelemahannya: pemetaan kunci → himpunan saham tidak mulus, sehingga perubahan kecil pada kunci bisa menukar saham. Ini justru bagian yang menguji kemampuan tiap algoritma.

3. Metaheuristik yang dibandingkan

Genetic Algorithm (GA). Populasi 100, inisialisasi acak (sebagian disemai dari heuristik top-K agar tidak mulai dari nol). Seleksi turnamen ukuran 3; crossover SBX ($p=0,9$, $\eta=15$) pada vektor 2N; mutasi Gaussian pada bobot mentah dan mutasi *swap* pada kunci (menukar prioritas satu saham terpilih dengan satu saham di luar); elitisme 2. Perbandingan individu memakai aturan Deb.

Simulated Annealing (SA). Satu solusi saja. Tetangga dibangkitkan dengan salah satu dari tiga gerakan: (i) tukar satu saham terpilih dengan saham di luar, (ii) geser bobot mentah antar dua saham terpilih, (iii) perturbasi Gaussian kecil. Penerimaan Metropolis dengan ΔE dihitung dari aturan Deb (pelanggaran diubah ke skala R). Pendinginan geometrik dari T0 (≈80% penerimaan awal) ke $0,001 \cdot T0$ dalam 20.000 iterasi.

Particle Swarm Optimization (PSO). Swarm 40, inersia 0,7, c1 = c2 = 1,5, 500 iterasi. Posisi = vektor 2N di atas, dibatasi ke rentang [0,1]. Kelebihan: sangat mudah diimplementasikan; kelemahan: rentan konvergen dini pada masalah kardinalitas.

Differential Evolution (DE). Populasi 100, strategi rand/1/bin, F=0,5, CR=0,9, 200 generasi. Operator tidak peka terhadap skala variabel sehingga cocok dengan campuran kunci dan bobot.

4. Anggaran evaluasi yang sama

Setiap metaheuristik diberi **20.000 evaluasi fungsi** per run (GA 100×200, SA 20.000 iterasi, PSO 40×500, DE 100×200). Konvergensi dicatat tiap 200 evaluasi untuk kurva perbandingan. Tiap kombinasi (algoritma, K, σ_max) dijalankan **10 kali (dinaikkan ke 30 bila waktu komputasi cukup)** dengan seed berbeda; seed yang sama dipakai antar algoritma agar inisialisasi dapat dibandingkan.

5. Ringkasan

Algoritma	**Jenis**	**Populasi**	**Operator kunci**	**Kekuatan yang diharapkan**	**Risiko**
Eksak (enumerasi)	Eksak	–	Enumerasi himpunan ($K \leq 5$)	Optimum terbukti	Tidak praktis untuk K = 7 dan 10 (27 juta dan 1,47 miliar himpunan)
Top-K + QP	Heuristik	–	Greedy forward (utama); urut μ lalu QP (pembanding)	Cepat, kuat	Bisa melewatkan kombinasi yang saling melindungi
GA	Evolusioner	100	SBX, mutasi, turnamen	Eksplorasi luas	Perlu penalaan operator
SA	Lintasan tunggal	1	Gerakan tukar saham	Sederhana, bagus untuk ruang kombinatorial	Sensitif jadwal pendinginan

PSO	Swarm	40	Kecepatan + memori	Cepat pada tahap awal	Konvergen dini
DE	Evolusioner	100	Mutasi diferensial	Tangguh terhadap skala	Kurang sesuai variabel diskrit

Hipotesis yang akan diuji, bukan diasumsikan: (H1) top-K + QP sangat kompetitif untuk K kecil, (H2) metaheuristik tertinggal dari solusi eksak ($K \leq 5$) atau solusi terbaik ($K = 7$ dan 10) dengan selisih yang melebar saat K membesar, dan (H3) tidak ada metaheuristik yang konsisten unggul pada semua K.

Skenario Eksperimen

Eksperimen dibagi dua fase: **Fase 1** (penalaan dan pemeriksaan kelayakan) dan **Fase 2** (perbandingan utama). Dengan begitu penalaan parameter tidak mencemari hasil akhir.

Fase 0 — Pemeriksaan kelayakan

Sebelum eksperimen, tentukan kombinasi (K, batas per saham, kas) yang secara matematis layak:

K	**Batas per saham**	**Maksimum terinvestasi**	**Status**
2, 3, 5	Tanpa batas	100%	Layak; bobot bebas, konsentrasi dibatasi oleh K dan batas risiko
7	20%	140%	Layak
10	20%	200%	Layak, batas 20% jarang mengikat

Batas 20% hanya dipakai untuk $K = 7$ dan 10, sehingga semua nilai K dapat terinvestasi penuh. Konsekuensinya, batas per saham tidak sama antar K: perbedaan hasil antara kelompok $K \leq 5$ dan $K \geq 7$ bercampur dengan perbedaan batas. Hal ini dinyatakan sebagai keterbatasan, dan efek diversifikasi dibaca terutama di dalam kelompok $K = 2$, 3, dan 5 yang memakai aturan sama. Sebagai kontrol, $K = 5$ juga dijalankan dengan batas 20% (bobot sama rata).

Fase 1 — Penalaan (hanya data latih jendela pertama)

- Tiap metaheuristik ditala pada satu konfigurasi tengah (K=7, σ_max = 1,5×), dengan grid kecil: GA (populasi × laju mutasi), SA (T0 × laju pendinginan), PSO (w × c), DE (F × CR).
- Setiap titik grid dijalankan 10 kali; pilih konfigurasi dengan median R tertinggi dan variansi terendah.
- Parameter terpilih **dibekukan** sebelum Fase 2.

Fase 2 — Perbandingan utama

Faktor penuh (factorial):

Faktor	**Level**
Kardinalitas K	2, 3, 5, 7, 10
Batas risiko σ_max	1 level: 1,5× volatilitas 1/N
Algoritma	Eksak (K ≤ 5), greedy forward, top-K+QP, acak valid, GA, SA, PSO, DE
Jendela walk-forward	3 jendela (latih 504, uji 126, geser 126 hari)
Run independen	10, dinaikkan ke 30 bila waktu cukup (hanya metaheuristik; baseline deterministik cukup 1)

Total sel: 5 K × 1 σ_max × 3 jendela = 15 masalah. Tiap masalah dijalankan oleh 4 metaheuristik × 10 run = 40 run, jadi 600 run × 20.000 evaluasi ≈ 12 juta evaluasi (bila run dinaikkan ke 30: 1.800 run, ≈ 36 juta evaluasi). Dengan evaluasi vektor (Σ berukuran 42×42) ini ringan; estimasi beberapa jam pada satu mesin biasa, lebih cepat jika dijalankan paralel. Untuk K = 2, 3, 5, pembanding eksak berupa enumerasi penuh (paling banyak 850.668 himpunan).

Desain tambahan (analisis sensitivitas)

- **Modal D**: satu level (Rp 5 juta). Variasi modal (Rp 5 juta sampai Rp 100 juta) ditunda sebagai pengembangan lanjutan.

- **Batas per saham pada K = 5**: dijalankan juga dengan batas 20% (bobot sama rata) sebagai kontrol terhadap K = 5 tanpa batas. Pada K < 5 batas 20% tidak layak sehingga tidak dijalankan.
- **Biaya transaksi**: 0 vs 0,25% per transaksi (opsional, jika waktu cukup).

Keluaran yang dicatat per run

R in-sample, σ in-sample, status kelayakan, himpunan saham terpilih, bobot, lot, kas, jumlah evaluasi sampai menemukan solusi terbaik, waktu komputasi, dan R, σ, Sharpe out-of-sample pada 126 hari uji. Semua disimpan ke CSV tunggal berformat long (satu baris per run) agar analisis statistik mudah.

Metode Evaluasi

Evaluasi dilakukan pada tiga lapis yang menjawab pertanyaan berbeda: seberapa baik algoritma **mengoptimalkan** (in-sample), seberapa baik hasilnya **bertahan** (out-of-sample), dan apakah perbedaannya **bermakna secara statistik**.

1. Kualitas optimasi (in-sample)

Metrik	**Definisi**	**Pertanyaan yang dijawab**
Optimality gap	$(R_{acuan} - R_{algo}) / R_{acuan}$	Seberapa jauh dari optimum sejati?
Tingkat kelayakan	Persentase run dengan $\sigma \leq \sigma_{max}$	Apakah algoritma menghormati kendala risiko?
Mean, median, std R	Atas 10 run	Seberapa baik dan seberapa konsisten?
Success rate	Persentase run dengan gap ≤ 1%	Seberapa sering menemukan solusi hampir optimal?
Evaluasi sampai terbaik	Rata-rata iterasi sampai solusi final ditemukan	Seberapa efisien?
Waktu komputasi	Detik per run	Biaya praktis

Kurva konvergensi	R terbaik vs jumlah evaluasi	Perilaku pencarian

R_acuan adalah optimum hasil enumerasi penuh untuk K = 2, 3, dan 5. Untuk K = 7 dan 10 tidak ada optimum terbukti, sehingga R_acuan adalah **solusi terbaik yang ditemukan oleh seluruh algoritma dan seluruh run**; gap pada K ini adalah gap terhadap yang terbaik yang diketahui, bukan terhadap optimum sejati, dan harus ditulis demikian di laporan.

2. Ketahanan hasil (out-of-sample, walk-forward)

Portofolio hasil optimasi pada jendela latih dipakai tanpa perubahan selama 126 hari uji berikutnya. Dicatat:

- Return tahunan, volatilitas tahunan, Sharpe, maximum drawdown, dan CVaR 95% harian pada data uji.
- **Realisasi risiko**: persentase jendela uji di mana σ realisasi $\leq \sigma_{max}$. Ini menguji apakah batas risiko yang ditetapkan pada data latih benar-benar terjaga di masa depan.
- **Penyusutan (shrinkage)**: R_uji − R_latih. Kita memperkirakan nilainya negatif besar karena optimasi memilih saham yang μ-nya kebetulan tinggi (estimation error). Algoritma yang mencari lebih keras bisa saja menyusut lebih parah; ini temuan yang layak dibahas.
- Turnover antar jendela (jika portofolio diperbarui) sebagai indikator kestabilan.

3. Uji statistik

- **Tahap ini: statistik deskriptif.** Laporkan mean, median, simpangan baku, kuartil, minimum, dan maksimum R serta gap per algoritma dan per K, dengan seed yang sama antar algoritma.
- **Tahap lanjutan: uji Wilcoxon signed-rank** berpasangan (mis. GA vs SA, GA vs greedy forward) pada median R per masalah, dengan koreksi Holm bila membandingkan lebih dari satu pasangan. Dengan 15 masalah dan 10 run, daya ujinya rendah, sehingga hasilnya dibaca sebagai indikasi, bukan bukti.
- Uji Friedman dan block bootstrap ditunda sebagai pengembangan lanjutan.
- Laporkan ukuran efek (selisih median), bukan hanya nilai-p.

4. Aturan penafsiran (ditetapkan sebelum melihat hasil)

1. Algoritma dianggap "setara" dengan solusi eksak (K = 2, 3, 5) bila gap median $\leq 1\%$.
2. Metaheuristik dianggap "bermanfaat" hanya bila secara konsisten mengalahkan greedy forward dan top-K+QP (uji signifikansi menyusul pada tahap lanjutan), atau bila mencapai gap serupa dengan waktu yang jauh lebih singkat pada K=10.
3. Perbedaan out-of-sample yang CI-nya mencakup nol tidak dilaporkan sebagai keunggulan.
4. Bila semua algoritma tertutup dalam gap $< 1\%$ pada $K \leq 5$ (hampir pasti, karena ruang pencarian hanya 861 sampai 850.668 himpunan dan dapat dienumerasi penuh), laporkan itu apa adanya: masalah kecil tidak membutuhkan metaheuristik. Perbandingan algoritma yang bermakna ada pada K = 7 dan 10.

5. Visualisasi yang direncanakan

Kurva konvergensi rata-rata ± std per algoritma; box plot R per K; heatmap gap (K × jendela) per algoritma; kurva R terhadap K dari solusi eksak ($K \leq 5$) berlapis titik-titik metaheuristik; diagram batang bobot terpilih untuk satu jendela contoh; kurva nilai kumulatif out-of-sample.

Rencana Implementasi

1. Perbaikan fondasi repo (lakukan lebih dulu)

Repo saat ini mengoptimalkan memakai log-return tetapi mengevaluasi memakai return sederhana (selisih $\mu \approx 7{,}6\%$/tahun), dan memakai harga yang sudah disesuaikan. Sebelum menambah algoritma, perbaiki:

#	**Perbaikan**	**Alasan**
1	Hitung μ dan Σ dari **return sederhana** harian, bukan log-return	Konsisten dengan fungsi tujuan dan evaluasi
2	Simpan **harga tak disesuaikan** (auto_adjust=False) untuk perhitungan lot; harga adjusted hanya untuk return	Lot dihitung dari harga nyata di bursa
3	Ganti MAX_WEIGHT dari 0,10 menjadi 0,20 (dipakai hanya untuk K = 7 dan 10) dan tambah K_LIST = [2, 3, 5, 7, 10], SIGMA_MULT = [1.5], D, LOT=100 di config.py	Parameter masalah baru
4	Buang entri "GA-terbaik" vs "GA-median" duplikat; ganti dengan distribusi 10 run	Perbandingan lebih jujur
5	Tambah jendela walk-forward menggantikan satu split latih/uji	Evaluasi lebih kuat

2. Struktur kode yang diusulkan

```
OptimasiPortofolioSaham/
  config.py              # parameter global
  main.py                # orkestrasi eksperimen
  src/
    scrappingdata.py     # unduh data (tambah harga raw)
    data.py              # muat harga, return sederhana, jendela walk-forward
    problem.py           # kelas Problem: mu, Sigma, harga, D, K, sigma_max
    decoder.py           # keys + raw weights -> portofolio layak (lot, cap, kas)
    fitness.py           # evaluasi (R, sigma, pelanggaran) + aturan Deb
    algorithms/
      exact_miqp.py      # enumerasi penuh K <= 5 (MISOCP opsional)
      topk_qp.py         # top-K + QP; greedy forward di greedy.py
      ga.py              # GA
      sa.py              # SA
      pso.py             # PSO
      de.py              # DE
      random_equal.py    # acak valid
    evaluasi.py          # metrik in/out-of-sample
    stats.py             # statistik deskriptif; Wilcoxon, Holm (tahap lanjutan)
    visualisasi.py
  tests/                 # pytest
  hasil/                 # CSV long + gambar
```

3. Antarmuka yang harus sama untuk semua algoritma

Semua algoritma menerima objek Problem dan anggaran evaluasi, lalu mengembalikan objek hasil yang sama.

```
@dataclass
class Problem:
    mu: np.ndarray       # (N,) return tahunan, return sederhana
    cov: np.ndarray      # (N,N) kovarians tahunan
```

```
    price: np.ndarray     # (N,) harga tak disesuaikan terakhir
    D: float              # modal (Rp)
    K: int
    sigma_max: float
    cap: float = 0.20
    lot: int = 100
    rf: float = 0.05

@dataclass
class Result:
    lots: np.ndarray      # (N,) integer
    R: float
    sigma: float
    feasible: bool
    history: list         # (n_eval, best_R_feasible) tiap 200 evaluasi
    n_eval: int
    seconds: float
    seed: int

def run(problem: Problem, budget: int, seed: int) -> Result: ...
```

4. Pseudo-code komponen inti

Decoder

```
def decode(x, P):
    keys, raw = x[:N], x[N:]
    S = np.argsort(-keys)[:P.K]
    w = np.zeros(N)
    w[S] = raw[S] / raw[S].sum()
    w = cap_redistribute(w, P.cap)          # iteratif, sisa -> kas
    lots = np.floor(w * P.D / (P.lot * P.price)).astype(int)
```

```
    spent = lots * P.lot * P.price
    w_final = spent / P.D                  # bobot nyata setelah pembulatan
    cash = 1 - w_final.sum()
    return lots, w_final, cash
```

Fitness dan perbandingan Deb

```
def evaluate(x, P):
    lots, w, cash = decode(x, P)
    sigma = np.sqrt(w @ P.cov @ w)
    R = w @ P.mu + cash * P.rf
    viol = max(0.0, sigma - P.sigma_max)
    return R, sigma, viol, lots


def better(a, b):
    # a, b = (R, sigma, viol, lots)
    if a.viol == 0 and b.viol == 0: return a.R > b.R
    if a.viol == 0: return True
    if b.viol == 0: return False
    return a.viol < b.viol
```

Solver eksak: enumerasi (K ≤ 5), MISOCP (cvxpy) opsional

```
w = cp.Variable(N); z = cp.Variable(N, boolean=True)
n = cp.Variable(N, integer=True)
constraints = [
    w == 100 * cp.multiply(P.price, n) / P.D,
    cp.sum(w) <= 1,
    w <= P.cap * z,
    n <= M * z,                # M = batas lot dari cap
    cp.sum(z) <= P.K,
    cp.norm(L.T @ w) <= P.sigma_max,   # L = cholesky(cov)
]
```

obj = cp.Maximize(P.mu @ w + P.rf * (1 - cp.sum(w)))

cp.Problem(obj, constraints).solve(solver=cp.SCIP, ...)

5. Urutan pengerjaan

Tahap	Pekerjaan	Keluaran	Cara memeriksa
1	Perbaikan fondasi (bagian 1)	Data return sederhana + harga raw	Cek μ dan Σ konsisten dengan evaluasi
2	problem.py, decoder.py, fitness.py	Evaluasi satu portofolio	Unit test: bobot $\leq 20\%$ ($K = 7$ dan 10), lot bulat, $\Sigma w + \text{kas} = 1$
3	Enumerasi eksak ($K \leq 5$), greedy forward, top-K+QP	Baseline	Bandingkan dengan brute force untuk K=2 (861 kombinasi)
4	GA	Metaheuristik pertama	Pada K=2 harus menemukan optimum brute force
5	SA, PSO, DE	Tiga pembanding	Pemeriksaan sama seperti GA
6	Fase 1 (penalaan)	Parameter beku	Hasil tercatat di CSV
7	Fase 2 (eksperimen utama)	CSV long 600 run	Tidak ada NaN; jumlah baris sesuai desain
8	Analisis dan visualisasi	Tabel dan gambar laporan	Reproduksi dengan satu perintah
9	Penulisan laporan	Draf akhir	Mengikuti struktur dokumen ini

6. Pengujian yang wajib

- **Brute force K=2 dan K=3**: metaheuristik harus menyamai atau mendekati optimum dari enumerasi lengkap (861 dan 11.480 kombinasi). Ini bukti paling kuat bahwa implementasi benar.
- **Invarian decoder**: untuk 10.000 vektor acak, bobot tidak pernah melebihi 20% (K = 7 dan 10), jumlah saham ≤ K, lot bilangan bulat, dan dana terpakai ≤ D.
- **Reproduksibilitas**: seed sama → hasil identik.
- **Kasus degenerat**: K = 1 (hanya untuk uji unit, bukan skenario penelitian) harus menghasilkan satu saham dengan seluruh dana terpakai sebanyak lot yang terjangkau (sisa hanya karena kelipatan lot).
- **Konsistensi solver eksak vs QP kontinu**: tanpa kardinalitas dan lot, hasil solver eksak harus sama dengan QP kontinu.

7. Hal teknis yang sering salah

- Jangan memanggil np.random global; buat np.random.default_rng(seed) per run.
- Kapitalisasi lot: IDX memakai lot 100 lembar; pembulatan ke bawah bisa membuat dana tidak terpakai sampai satu lot saham termahal (mis. harga Rp 10.000 → Rp 1 juta). Pada D kecil hal ini signifikan.
- Simpan Σ sekali per jendela dan jangan hitung ulang di setiap evaluasi.
- Σ harus semi-definit positif; periksa nilai eigen terkecil dan tambahkan regularisasi kecil (10^{-8}) jika perlu sebelum Cholesky.

Risiko, Keterbatasan, dan Referensi Awal

1. Risiko proyek dan mitigasinya

Risiko	**Dampak**	**Mitigasi**
Semua metaheuristik menyamai solver (gap < 1%)	Tidak ada "pemenang", terasa antiklimaks	Laporkan sebagai temuan: pada skala ini metaheuristik tidak diperlukan. Pertajam dengan skenario lebih sulit (modal kecil, biaya transaksi, K besar, universe lebih besar).
Tidak ada optimum terbukti untuk K = 7 dan 10	Tidak ada ground truth pada K besar	Laporkan gap terhadap solusi terbaik yang diketahui; bandingkan metaheuristik satu sama lain dan terhadap top-K+QP.

Hasil in-sample bagus tetapi out-of-sample buruk	Kesimpulan "optimal" tidak berguna	Sudah diantisipasi: ukur penyusutan, laporkan apa adanya, dan diskusikan estimation error.
Dosen mewajibkan GA saja	Perbandingan multi-algoritma dianggap di luar lingkup	Konfirmasi lebih awal. Jika benar, pertahankan SA (sudah ada di outline) dan solver eksak sebagai pembanding minimum.
Waktu komputasi membengkak	Jadwal mundur	Mulai dari K dan σ_max tengah; skala penuh hanya bila waktu cukup.
Data bermasalah (saham IPO, suspensi, split)	Return tidak valid	Pemeriksaan outlier return harian (> ±25%) dan catat saham yang dibuang beserta alasannya.

2. Keterbatasan penelitian

1. **Estimasi μ sangat bising.** Galat baku μ per saham median sekitar 18,7%/tahun pada 2 tahun data, dan tidak ada saham dengan $|t| > 2$. Sebagian besar "optimum" adalah optimum terhadap kebisingan. Solusi terbaik pada data latih tidak identik dengan solusi terbaik di masa depan.
2. **Bias penyintas (survivorship).** Universe diambil dari komponen LQ45 saat ini; saham yang pernah keluar dari indeks tidak ada. Hal ini cenderung menaikkan return historis.
3. **Empat saham dibuang** (AMMN, GOTO, MBMA, PGEO) karena data terlalu pendek; ini bias terhadap perusahaan lama.
4. **Biaya transaksi dan pajak** tidak ada dalam model dasar.
5. **Tanpa rebalancing dinamis** dalam satu jendela uji; portofolio dibeli sekali dan dipegang.
6. **Risk-free konstan** 5% walau suku bunga berubah.
7. **Dividen lewat harga adjusted** menganggap dividen langsung diinvestasikan ulang; tidak memodelkan arus kas dividen dan pajaknya.
8. **Ukuran risiko tunggal** (simpangan baku) tidak menangkap risiko ekor; CVaR bisa jadi perluasan.

3. Pengembangan lanjutan

- Pengganti pembatasan return murni: **multi-objektif NSGA-II** (R vs σ) untuk menghasilkan seluruh frontier dalam satu run.
- Estimator yang lebih stabil: shrinkage Ledoit–Wolf untuk Σ dan shrinkage James–Stein untuk μ, atau model faktor.

- Fungsi tujuan non-konveks yang membuat metaheuristik lebih relevan: Omega ratio, Calmar ratio, atau VaR historis.
- Rebalancing dinamis dengan biaya transaksi dan batas turnover.
- Universe lebih besar (mis. IDX80 atau seluruh saham IHSG yang likuid).

4. Referensi awal

> **Perhatian:** daftar berikut disusun dari ingatan dan **belum diverifikasi** satu per satu. Periksa judul, tahun, dan halaman pada sumber asli sebelum dimasukkan ke daftar pustaka laporan.

1. Markowitz, H. (1952). Portfolio Selection. *The Journal of Finance*, 7(1), 77–91.
2. Bienstock, D. (1996). Computational study of a family of mixed-integer quadratic programming problems. *Mathematical Programming*, 74, 121–140.
3. Chang, T.-J., Meade, N., Beasley, J. E., & Sharaiha, Y. M. (2000). Heuristics for cardinality constrained portfolio optimisation. *Computers & Operations Research*, 27(13), 1271–1296.
4. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
5. Deb, K., & Agrawal, R. B. (1995). Simulated binary crossover for continuous search space. *Complex Systems*, 9(2), 115–148.
6. Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.
7. Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95*, 1942–1948.
8. Storn, R., & Price, K. (1997). Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11, 341–359.
9. DeMiguel, V., Garlappi, L., & Uppal, R. (2009). Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy? *The Review of Financial Studies*, 22(5), 1915–1953.
10. Ledoit, O., & Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411.
11. Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.
12. Holm, S. (1979). A simple sequential rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
13. Charnes, A., & Cooper, W. W. (1962). Programming with linear fractional functionals. *Naval Research Logistics Quarterly*, 9(3–4), 181–186.